

Decision trees II: real inputs, real outputs, depth

Thresholds on sorted features, leaves that predict a mean, and the depth knob that decides how closely a tree fits.

THE QUESTION

How does a tree split a real-valued feature, predict a real number, and know when to stop growing?

THE INTUITION

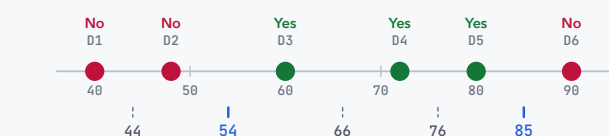
Sort the feature and try a threshold between each pair of neighbours. For numbers, purity means low spread, so score splits by MSE and let a leaf predict the mean. Every extra level fits the training rows more closely.

KEY TAKEAWAY

Same greedy recipe with two swaps: entropy or Gini for classes, MSE for numbers. Then limit depth or prune, and choose how much on validation data.

sort x → midpoints → score: entropy, Gini or MSE → keep the best → recurse → stop: depth, leaf size, pruning

1 A threshold makes a number a yes/no test



5 midpoints = 5 distinct splits; blue: the label changes

Temperature could split at 45, 45.1, 45.01 and so on, but every threshold between two neighbouring sorted values gives the same partition. So sort, and try the $n - 1$ midpoints $(x_i + x_{i+1})/2$. The slides shorten the list further: only midpoints where the label changes (54 and 85) can win on entropy.

2 Each threshold is scored like an attribute

split	left	right	weighted H	gain = 1 - weighted H
≤ 44	●	●●●●	0.809	0.191
≤ 54	●●	●●●●	0.541	0.459
≤ 66	●●●	●●●	0.918	0.082
≤ 76	●●●●	●	1.000	0.000
≤ 85	●●●●●		0.809	0.191

parent: 3 Yes, 3 No, H = 1.000 bit

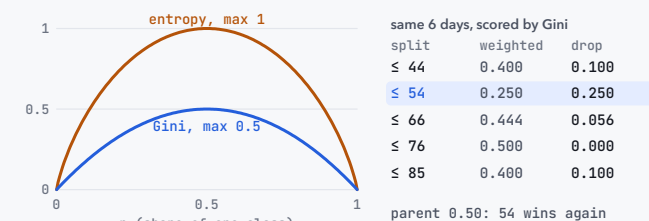
dots: No / Yes per side

A test $x \leq t$ is a two-valued attribute, so the gain from the decision trees I sheet applies:

$$\text{Gain} = H(S) - \frac{|S_L|}{|S|} H(S_L) - \frac{|S_R|}{|S|} H(S_R)$$

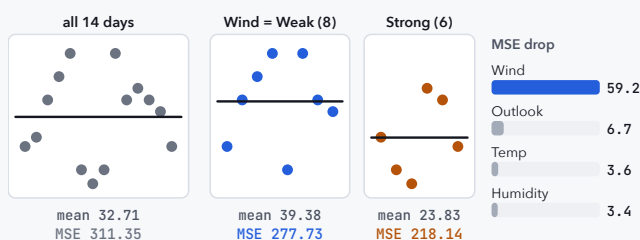
54 wins: the left side is pure and the right is 3 Yes, 1 No (0.811). The next split, at 85, makes every leaf pure. The slides' table uses a parent entropy of 0.950 and gives 0.650 at 85; they should be 1.000 and 0.809.

3 Gini impurity: no logs, same ranking here



Gini(S) = $1 - \sum_k p_k^2$. Like entropy it is 0 for a pure node and largest at an even mix (0.5 for two classes). It skips the log, so it is a little cheaper, and it is scikit-learn's default criterion. Here both criteria rank the five thresholds the same way.

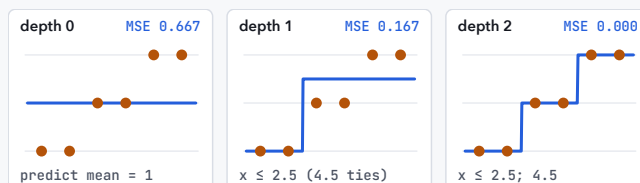
4 Real outputs: impurity becomes MSE



$$\text{MSE}(S) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

This is the variance of the node's targets. A leaf predicts the mean $\bar{y}$, the constant with the smallest squared error. The tennis table with minutes played: $311.35 \rightarrow \frac{8}{14} \cdot 277.73 + \frac{6}{14} \cdot 218.14 = 252.19$, a drop of 59.15 (19%), so Wind goes first. The slides print 252.18 and 59.17 after rounding mid-way.

5 Real in, real out: minimise weighted MSE



With $R_1 = \{x \leq s\}$ and $R_2 = \{x > s\}$, pick

$$s^* = \arg \min_s \frac{|R_1|}{n} \text{MSE}(R_1) + \frac{|R_2|}{n} \text{MSE}(R_2)$$

over the midpoints. On $x = 1, \dots, 6$ with $y = 0, 0, 1, 1, 2, 2$ the thresholds 2.5 and 4.5 tie; scikit-learn keeps the first. The prediction is a staircase.

6 More depth gives a finer staircase

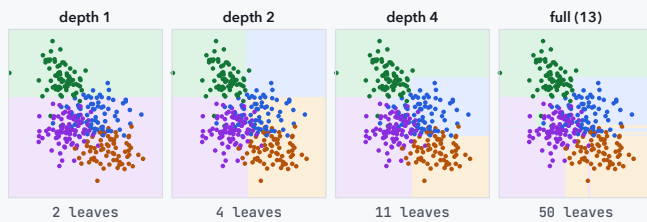


200 points of $\sin x$ on $[0, 2\pi]$ (orange). Depth 1 splits at π with leaves ± 0.633 ; MSE falls from 0.4975 to 0.0963. With no depth limit every point gets its own leaf: 200 leaves, depth 20.

Depth decides how closely a tree fits

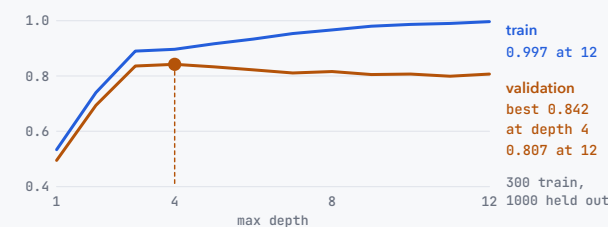
300 points from four overlapping blobs (a re-creation of the notebook's data, seed 0), 1000 more held out.

7 Splits are axis-aligned, so leaves are boxes



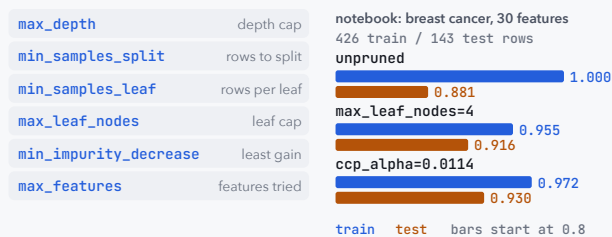
A test $x_j \leq t$ cuts along a line parallel to an axis, so the tree tiles the plane with one box per leaf. Depth 1 can name only two of the four classes. With no limit the tree splits until every leaf is pure: 50 leaves and thin slivers around single points.

8 Held-out accuracy peaks, then slips



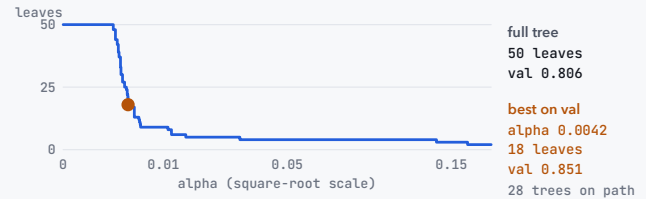
Training accuracy climbs to 1.000 for the full tree. Held-out accuracy peaks at depth 4 (0.842) and slips to 0.806: the extra leaves fit noise. That gap is overfitting, the subject of the bias and variance sheet. Choose the depth on validation data or by cross-validation.

9 Pre-pruning stops the tree early



The scikit-learn knobs on the left each stop a split before it happens. They are simple and cheap, but can stop before a useful split lower down. On the blobs, `min_samples_leaf=20` gives 10 leaves and 0.828 held out. Bars: the pruning notebook's numbers.

10 Post-pruning cuts the weakest link

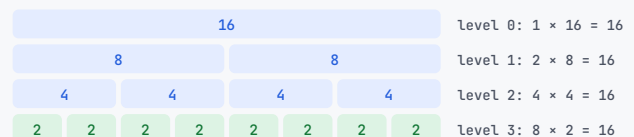


Grow fully, then minimise $R_\alpha(T) = R(T) + \alpha |T|$, with R the leaf impurity weighted by share of rows and $|T|$ the leaf count. Node t is cut once α passes

$$g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}.$$

Cut the smallest g , repeat to the root, pick α on validation data.

11 Training is $O(MN \log N)$; prediction is $O(h)$



N rows, M features, depth h . Presort each feature once, $O(MN \log N)$; children inherit the order, and each level holds all N rows, so the sweeps cost $O(MNh)$. Balanced, that is $O(MN \log N)$; sorting again at every node gives $O(MN(\log N)^2)$. A prediction walks one path, $O(h)$: $\log N$ to N .

12 The linear sweep in NumPy

```
def best_threshold(x, y):
    o = np.argsort(x); x, y = x[o], y[o]
    n = len(y); k = np.arange(1, n)
    s1 = np.cumsum(y)[:n-1] # left sums
    s2 = np.cumsum(y**2)[:n-1] # of y, y^2
    sse_l = s2 - s1**2 / k
    r1, r2 = y.sum() - s1, y @ y - s2
    sse_r = r2 - r1**2 / (n - k)
    loss = sse_l + sse_r
    loss[x[1:] == x[:n-1]] = np.inf # tie
    i = np.argmin(loss)
    return (x[i]+x[i+1]) / 2, loss[i] / n
```

Running sums give $SSE = \sum y^2 - (\sum y)^2 / n$ per side in $O(1)$ (for classes, keep label counts). Sine data: 3.142, 0.0963.

13 Pause and think

Temperature in Fahrenheit? Same splits: only the order of values matters.

Thresholds for n distinct values? $n - 1$; any other repeats one of their partitions.

Why does a leaf predict the mean? It has the least squared error; the leaf's MSE is then its variance.

Can depth 1 separate four classes? No: two leaves, two classes (0.533 train accuracy).

Weighted points (boosting)? Sum weights for p . Slides: $H = 1$, then 0.971 per side, gain 0.029.

Pick depth by test accuracy? No, that tunes on the test set; use a validation split.

try midpoints · Gini or entropy for classes · MSE and the mean for numbers · tune depth on validation data

Before: the decision trees I sheet; next: the bias and variance sheet. Source: lecture slides on decision trees, pruning and their time complexity, and the decision-tree notebooks ([nipunbatra.github.io/ml-teaching](https://github.com/nipunbatra/ml-teaching)); breast-cancer bars quoted from the pruning notebook.